

Pure Past Linear Temporal Logic

Game-theoretic Approach to LTLf Planning and Synthesis

Giuseppe De Giacomo



- 1 PPLTL: Pure Past LTL
- 2 PPLTL and LTL_f
- 3 PPLTL to Automata
- 4 PPLTL Reasoning and Synthesis
- 5 PPLTL Again
- 6 PPLTL to Automata (Revisited)

PPLTL: the language

Syntax analogous to LTL_f but interpreted backwards

$$\varphi ::= A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \supset \varphi_2 \mid \Theta\varphi \mid \Theta\varphi \mid \Diamond\varphi \mid \Box\varphi \mid \varphi_1 \mathcal{S} \varphi_2 \\ Y\varphi \mid WY\varphi \mid O\varphi \mid H\varphi \mid \varphi_1 \mathcal{S} \varphi_2 \quad (\text{alternative notation})$$

- A : **atomic** propositions
- $\neg\varphi, \varphi_1 \wedge \varphi_2, \varphi_1 \vee \varphi_2, \varphi_1 \supset \varphi_2$: **boolean** connectives
- $\Theta\varphi$: “**previous step exists** and at **previous** step (of the trace) φ holds” (Y)esterday φ
- $\Theta\varphi$: “**if previous step exists** then at **previous** step φ ” ($\Theta\varphi \equiv \neg\Theta\neg\varphi$) (WY)esterday φ
- $\Diamond\varphi$: “ φ **sometime in the past**” ($\Diamond\varphi \equiv \text{true} \mathcal{S} \varphi$) (O)nce φ
- $\Box\varphi$: “**always in the past**” ($\Box\varphi \equiv \neg\Diamond\neg\varphi$) (H)istorically φ
- $\varphi_1 \mathcal{S} \varphi_2$: “**sometime in the past** φ_2 and **since** then (not including that instant) φ_1 holds” φ_1 (S)ince φ_2

“To enter a club one must have been accepted as a member”

$$\Box(\text{EnterDangerousArea} \supset \Diamond(\text{Permission})) \quad H(\text{EnterDangerousArea} \supset O(\text{Permission}))$$

“Every time you took the bus, you bought a new ticket beforehand”

$$\Box(\text{TakeBus} \supset \Theta(\neg\text{TakeBus} \mathcal{S} \text{BuyTicket})) \quad H(\text{TakeBus} \supset Y(\neg\text{TakeBus} \mathcal{S} \text{BuyTicket}))$$

PPLTL: the language

Syntax analogous to LTL_f but interpreted backwards

$$\varphi ::= A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \supset \varphi_2 \mid \Theta\varphi \mid \Theta\varphi \mid \Diamond\varphi \mid \Box\varphi \mid \varphi_1 \mathcal{S} \varphi_2 \\ Y\varphi \mid WY\varphi \mid O\varphi \mid H\varphi \mid \varphi_1 \mathcal{S} \varphi_2 \quad (\text{alternative notation})$$

- A : **atomic** propositions
- $\neg\varphi, \varphi_1 \wedge \varphi_2, \varphi_1 \vee \varphi_2, \varphi_1 \supset \varphi_2$: **boolean** connectives
- $\Theta\varphi$: “**previous step exists** and at **previous** step (of the trace) φ holds” (Y)esterday φ
- $\Theta\varphi$: “**if previous step exists** then at **previous** step φ ” ($\Theta\varphi \equiv \neg\Theta\neg\varphi$) (WY)esterday φ
- $\Diamond\varphi$: “ φ **sometime in the past**” ($\Diamond\varphi \equiv \text{true} \mathcal{S} \varphi$) (O)nce φ
- $\Box\varphi$: “**always in the past**” ($\Box\varphi \equiv \neg\Diamond\neg\varphi$) (H)istorically φ
- $\varphi_1 \mathcal{S} \varphi_2$: “**sometime in the past** φ_2 and **since** then (not including that instant) φ_1 holds” φ_1 ($\mathcal{S}$)ince φ_2

“To enter a club one must have been accepted as a member”

$$\Box(\text{EnterDangerousArea} \supset \Diamond(\text{Permission})) \quad H(\text{EnterDangerousArea} \supset O(\text{Permission}))$$

“Every time you took the bus, you bought a new ticket beforehand”

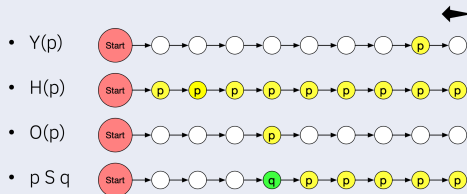
$$\Box(\text{TakeBus} \supset \Theta(\neg\text{TakeBus} \mathcal{S} \text{BuyTicket})) \quad H(\text{TakeBus} \supset Y(\neg\text{TakeBus} \mathcal{S} \text{BuyTicket}))$$

PPLTL semantics

Given a finite trace π , we inductively define when an PPLTL formula φ is **true** at an instant i (for $0 \leq i \leq \text{last}$), where *last* is the current moment from which we evaluate the formula, in symbols $\pi, i \models \varphi$, as follows:

- $\pi, i \models A$, for $A \in \mathcal{P}$ iff $A \in \pi(i)$.
- $\pi, i \models \neg \varphi$ iff $\pi, i \not\models \varphi$.
- $\pi, i \models \varphi_1 \wedge \varphi_2$ iff $\pi, i \models \varphi_1$ and $\pi, i \models \varphi_2$.
- $\pi, i \models \Theta \varphi$ iff $0 \leq i-1$ and $\pi, i-1 \models \varphi$.
- $\pi, i \models \Theta \varphi$ iff $0 \leq i-1$ implies $\pi, i-1 \models \varphi$.
- $\pi, i \models \Diamond \varphi$ iff for some j s.t. $0 \leq j \leq i$, we have $\pi, j \models \varphi$.
- $\pi, i \models \Box \varphi$ iff for all j s.t. $0 \leq j \leq i$, we have $\pi, j \models \varphi$.
- $\pi, i \models \varphi_1 \mathcal{S} \varphi_2$ iff for some j s.t. that $0 \leq j \leq i$ we have $\pi, j \models \varphi_2$ and for all k , $j < k \leq i$ we have $\pi, k \models \varphi_1$.

Past operators in pictures



Let a, b be propositional formulas.

Template	PPLTL Formula	LTL_f Formula
$\text{init}(a)$	$\Diamond(a \wedge \neg \Theta \text{true})$	a
$\text{last}(a)$	a	$\Diamond(a \wedge \neg \bigcirc \text{true})$

Templates from: Matthew B. Dwyer, George S. Avrunin, James C. Corbett. Patterns in Property Specifications for Finite-State Verification. ICSE1999.
Adopted in BPM: Wil M. P. van der Aalst, Maja Pesic, Helen Schonenberg. Declarative workflows: Balancing between Flexibility and Support. CSRD 2009.

Let a, b be propositional formulas.

Template	PPLTL Formula	LTL_f Formula
existence(a)	$\Diamond a$	$\Diamond a$
absence(a)	$\Box \neg a$	$\Box \neg a$
absence2(a)	$\Box(a \rightarrow \Theta \Box \neg a)$	$\Box(a \supset \bullet \Box \neg a)$
choice(a, b)	$\Diamond a \vee \Diamond b$	$\Diamond a \vee \Diamond b$
exclusive-choice(a, b)	$(\Diamond a \vee \Diamond b) \wedge (\Box \neg a \vee \Box \neg b)$	$(\Diamond a \vee \Diamond b) \wedge (\Box \neg a \wedge \Box \neg b)$
co-existence(a, b)	$\Diamond a \leftrightarrow \Diamond b$	$\Diamond a \leftrightarrow \Diamond b$
responded-existence(a, b)	$\Diamond a \rightarrow \Diamond b$	$\Diamond a \rightarrow \Diamond b$
not-co-existence(a, b)	$\Diamond a \rightarrow \Box \neg b$	$\Diamond a \rightarrow \Box \neg b$

Templates from: Matthew B. Dwyer, George S. Avrunin, James C. Corbett. Patterns in Property Specifications for Finite-State Verification. ICSE1999.
 Adopted in BPM: Wil M. P. van der Aalst, Maja Pesic, Helen Schonenberg. Declarative workflows: Balancing between Flexibility and Support. CSRD 2009.

Let a, b be propositional formulas.

Template	PPLTL Formula	LTL _f Formula
response(a, b)	$(\neg a \mathcal{S} b) \vee \Box \neg a$	$\Box(a \rightarrow \Diamond b)$
precedence(a, b)	$\Box(b \rightarrow \Diamond a)$	$(\neg b \mathcal{U} a) \vee \Box \neg b$
chain-response(a, b)	$\Box(\Theta a \rightarrow b) \wedge \neg a$	$\Box(a \rightarrow \bigcirc b)$
chain-precedence(a, b)	$\Box(b \rightarrow \Theta a)$	$\Box(\bigcirc b \rightarrow a) \wedge \neg b$
not-succession(a, b)	$\Box(b \rightarrow \Box \neg a)$	$\Box(a \rightarrow \Box \neg b)$
not-chain-succession(a, b)	$\Box(b \rightarrow \Theta \neg a)$	$\Box(a \rightarrow \bullet \neg b)$

Templates from: Matthew B. Dwyer, George S. Avrunin, James C. Corbett. Patterns in Property Specifications for Finite-State Verification. ICSE1999.
 Adopted in BPM: Wil M. P. van der Aalst, Maja Pesic, Helen Schonenberg. Declarative workflows: Balancing between Flexibility and Support. CSRD 2009.

PPLTL to FOL

We can translate any PPLTL formula to FOL

- $fol(A, x) = A(x)$
- $fol(\neg\varphi, x) = \neg fol(\varphi, x)$
- $fol(\varphi_1 \wedge \varphi_2, x) = fol(\varphi_1, x) \wedge fol(\varphi_2, x)$
- $fol(\Theta\varphi, x) = \exists y. succ(y, x) \wedge fol(\varphi, y)$
- $fol(\varphi \mathcal{S} \varphi', x) = \exists y. 0 \leq y \leq x \wedge fol(\varphi', y) \wedge \forall z. y < z \leq x \rightarrow fol(\varphi, z)$

Example

$\exists (EnterDangerousArea \supset \Diamond Permission)$ is translated to
 $\forall x. EnterDangerousArea(x) \supset \exists y. 0 \leq y \leq x \wedge Permission(y)$

And viceversa!

Theorem ([GabbayPnueliShelahStavi80] – see also [Kamp68])

PPLTL has the same expressive power of FOL, i.e., the same expressive power of LTL_f .

PPLTL to FOL

We can translate any PPLTL formula to FOL

- $fol(A, x) = A(x)$
- $fol(\neg\varphi, x) = \neg fol(\varphi, x)$
- $fol(\varphi_1 \wedge \varphi_2, x) = fol(\varphi_1, x) \wedge fol(\varphi_2, x)$
- $fol(\Theta\varphi, x) = \exists y. succ(y, x) \wedge fol(\varphi, y)$
- $fol(\varphi \mathcal{S} \varphi', x) = \exists y. 0 \leq y \leq x \wedge fol(\varphi', y) \wedge \forall z. y < z \leq x \rightarrow fol(\varphi, z)$

Example

$\exists (EnterDangerousArea \supset \Diamond Permission)$ is translated to
 $\forall x. EnterDangerousArea(x) \supset \exists y. 0 \leq y \leq x \wedge Permission(y)$

And viceversa!

Theorem ([GabbayPnueliShelahStavi80] – see also [Kamp68])

PPLTL has the same expressive power of FOL, i.e., the same expressive power of LTL_f .

PPLTL to FOL

We can translate any PPLTL formula to FOL

- $fol(A, x) = A(x)$
- $fol(\neg\varphi, x) = \neg fol(\varphi, x)$
- $fol(\varphi_1 \wedge \varphi_2, x) = fol(\varphi_1, x) \wedge fol(\varphi_2, x)$
- $fol(\Theta\varphi, x) = \exists y. succ(y, x) \wedge fol(\varphi, y)$
- $fol(\varphi \mathcal{S} \varphi', x) = \exists y. 0 \leq y \leq x \wedge fol(\varphi', y) \wedge \forall z. y < z \leq x \rightarrow fol(\varphi, z)$

Example

$\exists (EnterDangerousArea \supset \Diamond Permission)$ is translated to
 $\forall x. EnterDangerousArea(x) \supset \exists y. 0 \leq y \leq x \wedge Permission(y)$

And viceversa!

Theorem ([GabbayPnueliShelahStavi80] – see also [Kamp68])

PPLTL *has the same expressive power of FOL, i.e., the same expressive power of* LTL_f .

- 1 PPLTL: Pure Past LTL
- 2 **PPLTL and LTL_f**
- 3 PPLTL to Automata
- 4 PPLTL Reasoning and Synthesis
- 5 PPLTL Again
- 6 PPLTL to Automata (Revisited)

PPLTL

Syntax analogous to LTL_f but interpreted backwards

$$\begin{aligned} \varphi ::= & A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \supset \varphi_2 \mid \\ & \Theta\varphi \mid \Theta\varphi \mid \Diamond\varphi \mid \Box\varphi \mid \varphi_1 S \varphi_2 \\ & Y\varphi \mid WY\varphi \mid O\varphi \mid H\varphi \mid \varphi_1 S \varphi_2 \quad (\text{alternative notation}) \end{aligned}$$

- A : **atomic** propositions
- $\neg\varphi, \varphi_1 \wedge \varphi_2, \varphi_1 \vee \varphi_2, \varphi_1 \supset \varphi_2$: **boolean** connectives
- $\Theta\varphi$: “**previous step exists** and at **previous** step (of the trace) φ holds”
- $\Theta\varphi$: “if **previous step exists** then at **previous** step φ ”
($\Theta\varphi \equiv \neg\Theta\neg\varphi$)
- $\Diamond\varphi$: “ φ **eventually in the past**” ($\Diamond\varphi \equiv \text{true } S \varphi$)
- $\Box\varphi$: “**always in the past**” ($\Box\varphi \equiv \neg\Diamond\neg\varphi$)
- $\varphi_1 S \varphi_2$: “**eventually in the past** φ_2 and **since** then φ_1 holds (not including that instant)”

LTL_f

Same syntax as standard LTL but interpreted over finite traces

$$\begin{aligned} \varphi ::= & A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \supset \varphi_2 \mid \\ & \bigcirc\varphi \mid \bullet\varphi \mid \Diamond\varphi \mid \Box\varphi \mid \varphi_1 \mathcal{U} \varphi_2 \\ & X\varphi \mid WX\varphi \mid F\varphi \mid G\varphi \mid \varphi_1 U \varphi_2 \quad (\text{alternative notation}) \end{aligned}$$

- A : **atomic** propositions
- $\neg\varphi, \varphi_1 \wedge \varphi_2, \varphi_1 \vee \varphi_2, \varphi_1 \supset \varphi_2$: **boolean** connectives
- $\bigcirc\varphi$: “(**next step exists** and) at **next** step (of the trace) φ holds”
- $\bullet\varphi$: “if **next step exists** then at **next** step φ holds” (*weak next*)
($\bullet\varphi \equiv \neg\bigcirc\neg\varphi$)
- $\Diamond\varphi$: “ φ will **eventually** hold” ($\Diamond\varphi \equiv \text{true } \mathcal{U} \varphi$)
- $\Box\varphi$: “from current till last instant φ will **always** hold”
($\Box\varphi \equiv \neg\Diamond\neg\varphi$)
- $\varphi_1 \mathcal{U} \varphi_2$: “**eventually** φ_2 holds, and φ_1 holds **until** φ_2 does (not including that instant)”

PPLTL and LTL_f have exactly the same expressive power, that of FOL over finite traces.

PPLTL

Syntax analogous to LTL_f but interpreted backwards

$$\begin{aligned} \varphi ::= & A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \supset \varphi_2 \mid \\ & \Theta\varphi \mid \Theta\varphi \mid \Diamond\varphi \mid \Box\varphi \mid \varphi_1 S \varphi_2 \\ & Y\varphi \mid WY\varphi \mid O\varphi \mid H\varphi \mid \varphi_1 S \varphi_2 \quad (\text{alternative notation}) \end{aligned}$$

- A : **atomic** propositions
- $\neg\varphi, \varphi_1 \wedge \varphi_2, \varphi_1 \vee \varphi_2, \varphi_1 \supset \varphi_2$: **boolean** connectives
- $\Theta\varphi$: “**previous step exists** and at **previous** step (of the trace) φ holds”
- $\Theta\varphi$: “if **previous step exists** then at **previous** step φ ”
($\Theta\varphi \equiv \neg\Theta\neg\varphi$)
- $\Diamond\varphi$: “ φ **eventually in the past**” ($\Diamond\varphi \equiv \text{true } S \varphi$)
- $\Box\varphi$: “**always in the past**” ($\Box\varphi \equiv \neg\Diamond\neg\varphi$)
- $\varphi_1 S \varphi_2$: “**eventually in the past** φ_2 and **since** then φ_1 holds (not including that instant)”

LTL_f

Same syntax as standard LTL but interpreted over finite traces

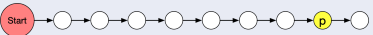
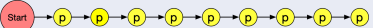
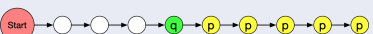
$$\begin{aligned} \varphi ::= & A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \supset \varphi_2 \mid \\ & \bigcirc\varphi \mid \bullet\varphi \mid \Diamond\varphi \mid \Box\varphi \mid \varphi_1 U \varphi_2 \\ & X\varphi \mid WX\varphi \mid F\varphi \mid G\varphi \mid \varphi_1 U \varphi_2 \quad (\text{alternative notation}) \end{aligned}$$

- A : **atomic** propositions
- $\neg\varphi, \varphi_1 \wedge \varphi_2, \varphi_1 \vee \varphi_2, \varphi_1 \supset \varphi_2$: **boolean** connectives
- $\bigcirc\varphi$: “(**next step exists** and) at **next** step (of the trace) φ holds”
- $\bullet\varphi$: “if **next step exists** then at **next** step φ holds” (*weak next*)
($\bullet\varphi \equiv \neg\bigcirc\neg\varphi$)
- $\Diamond\varphi$: “ φ will **eventually** hold” ($\Diamond\varphi \equiv \text{true } U \varphi$)
- $\Box\varphi$: “from current till last instant φ will **always** hold”
($\Box\varphi \equiv \neg\Diamond\neg\varphi$)
- $\varphi_1 U \varphi_2$: “**eventually** φ_2 holds, and φ_1 holds **until** φ_2 does (not including that instant)”

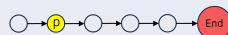
PPLTL and LTL_f have exactly the same expressive power, that of FOL over finite traces.

PPLTL and LTL_f have exactly the same expressive power, that of FOL over finite traces.

PPLTL

- $Y(p)$ 
- $H(p)$ 
- $O(p)$ 
- $p \text{ S } q$ 

LTL_f

- $X(p)$ 
- $G(p)$ 
- $F(p)$ 
- $p \text{ U } q$ 

PPLTL and LTL_f have exactly the same expressive power, that of FOL over finite traces.

PPLTL to FOL

We can translate any PPLTL formula to FOL (and viceversa)

- $fol(A, x) = A(x)$
- $fol(\neg\varphi, x) = \neg fol(\varphi, x)$
- $fol(\varphi_1 \wedge \varphi_2, x) = fol(\varphi_1, x) \wedge fol(\varphi_2, x)$
- $fol(\Theta\varphi, x) = \exists y. succ(y, x) \wedge fol(\varphi, y)$
- $fol(\varphi \mathcal{S} \varphi', x) = \exists y. 0 \leq y \leq x \wedge fol(\varphi', y) \wedge \forall z. y < z \leq x \supset fol(\varphi, z)$

For example

$\exists (EnterDangerousArea \supset \Diamond Permission)$ is translated to
 $\forall x. EnterDangerousArea(x) \supset \exists y. 0 \leq y \leq x \wedge Permission(y)$

LTL_f to FOL

We can translate any LTL_f formula to FOL (and viceversa)

- $fol(A, x) = A(x)$
- $fol(\neg\varphi, x) = \neg fol(\varphi, x)$
- $fol(\varphi_1 \wedge \varphi_2, x) = fol(\varphi_1, x) \wedge fol(\varphi_2, x)$
- $fol(\bigcirc\varphi, x) = \exists y. succ(x, y) \wedge fol(\varphi, y)$
- $fol(\varphi \mathcal{U} \varphi', x) = \exists y. x \leq y \leq last \wedge fol(\varphi', y) \wedge \forall z. x \leq z < y \supset fol(\varphi, z)$

For example

$\square (LowPwr \supset \Diamond Recharged)$ is translated to
 $\forall x. LowPwr(x) \supset \exists y. x \leq y \leq last \wedge Recharged(y)$

PPLTL and LTL_f have exactly the same expressive power, that of FOL over finite traces.

How easy is to translate PPLTL to LTL_f ?

- PPLTL and LTL_f are both exponentially more succinct than the other [Artale et al. TIME2023]
- Best systematic translation known is 3-EXPTIME [De Giacomo et al. IJCAI2020]
- Systematic exponential procedure exists for fragments (no until/since but with eventually/once) [Artale et al. KR2023]
- For some simple formulas one can do it manually by exploiting the semantics (see Declare and PDDL3 formulas for which the translation is linear [GeattiMontaliRivkinAAAI2024]).
- In general, use PPLTL or LTL_f depending on which is more suited to express your desired specifications

PPLTL and LTL_f have exactly the same expressive power, that of FOL over finite traces.

How easy is to translate PPLTL to LTL_f ?

- PPLTL and LTL_f are both exponentially more succinct than the other [Artale et al. TIME2023]
- Best systematic translation known is 3-EXPTIME [De Giacomo et al. IJCAI2020]
- Systematic exponential procedure exists for fragments (no until/since but with eventually/once) [Artale et al. KR2023]
- For some simple formulas one can do it manually by exploiting the semantics (see Declare and PDDL3 formulas for which the translation is linear [GeattiMontaliRivkinAAAI2024]).
- In general, use PPLTL or LTL_f depending on which is more suited to express your desired specifications

- 1 PPLTL: Pure Past LTL
- 2 PPLTL and LTL_f
- 3 PPLTL to Automata**
- 4 PPLTL Reasoning and Synthesis
- 5 PPLTL Again
- 6 PPLTL to Automata (Revisited)

Key point

PPLTL formulas can be translated into a DFA $\mathcal{A}_\varphi$ such that $t \models \varphi$ iff $t \in \mathcal{L}(\mathcal{A}_\varphi)$.

(as for LTL_f)

How

Basic algorithm for computing a DFA $\mathcal{A}_\varphi$ from a PPLTL formula φ :

- Compute an AFA $\mathcal{A}_\varphi^-$ that recognize the trace **starting from the end** – (linear time)
(this AFA is **exactly** the one for LTL_f, except for the syntactic use of the past operator symbols)
- Compute the DFA $\mathcal{A}_\varphi$ for the **reverse language** of the AFA $\mathcal{A}_\varphi^-$ – (exponential time)
(this can be done by standard language theoretic techniques, see, e.g., [DeGiacomoEtAlIJCAI2020])
- (Optional) Minimize the DFA $\mathcal{A}_\varphi$ to obtain smaller DFA in canonical form – (polynomial time)
(recall there is only one minimal DFA for each regular language)

PPLTL to DFA



LTL_f to DFA



Key point

PPLTL formulas can be translated into a DFA $\mathcal{A}_\varphi$ such that $t \models \varphi$ iff $t \in \mathcal{L}(\mathcal{A}_\varphi)$.

(as for LTL_f)

How

Basic algorithm for computing a DFA $\mathcal{A}_\varphi$ from a PPLTL formula φ :

- Compute an AFA $\mathcal{A}_\varphi^-$ that recognize the trace **starting from the end** – (linear time)
(this AFA is **exactly** the one for LTL_f, except for the syntactic use of the past operator symbols)
- Compute the DFA $\mathcal{A}_\varphi$ for the **reverse language** of the AFA $\mathcal{A}_\varphi^-$ – (exponential time)
(this can be done by standard language theoretic techniques, see, e.g., [DeGiacomoEtAlIJCAI2020])
- (Optional) Minimize the DFA $\mathcal{A}_\varphi$ to obtain smaller DFA in canonical form – (polynomial time)
(recall there is only one minimal DFA for each regular language)

PPLTL to DFA



LTL_f to DFA



Key point

PPLTL formulas can be translated into a DFA $\mathcal{A}_\varphi$ such that $t \models \varphi$ iff $t \in \mathcal{L}(\mathcal{A}_\varphi)$.

(as for LTL_f)

How

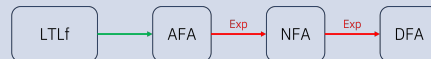
Basic algorithm for computing a DFA $\mathcal{A}_\varphi$ from a PPLTL formula φ :

- Compute an AFA $\mathcal{A}_\varphi^-$ that recognize the trace **starting from the end** – (linear time)
(this AFA is **exactly** the one for LTL_f, except for the syntactic use of the past operator symbols)
- Compute the DFA $\mathcal{A}_\varphi$ for the **reverse language** of the AFA $\mathcal{A}_\varphi^-$ – (exponential time)
(this can be done by standard language theoretic techniques, see, e.g., [DeGiacomoEtAlIJCAI2020])
- (Optional) Minimize the DFA $\mathcal{A}_\varphi$ to obtain smaller DFA in canonical form – (polynomial time)
(recall there is only one minimal DFA for each regular language)

PPLTL to DFA



LTL_f to DFA



- 1 PPLTL: Pure Past LTL
- 2 PPLTL and LTL_f
- 3 PPLTL to Automata
- 4 **PPLTL Reasoning and Synthesis**
- 5 PPLTL Again
- 6 PPLTL to Automata (Revisited)

PPLTL Satisfiability (φ SAT)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for φ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

PPLTL Validity (φ VAL)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for $\neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

PPLTL Logical Implication ($\Gamma \models \varphi$)

- 1: Given PPLTL formulas Γ and φ
- 2: Compute reverse AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (DFA can be constructed while checking nonemptiness.)
As for LTL_f and LTL on infinite traces.

PPLTL Satisfiability (φ SAT)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for φ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

PPLTL Validity (φ VAL)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for $\neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

PPLTL Logical Implication ($\Gamma \models \varphi$)

- 1: Given PPLTL formulas Γ and φ
- 2: Compute reverse AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (DFA can be constructed while checking nonemptiness.)
As for LTL_f and LTL on infinite traces.

PPLTL Satisfiability (φ SAT)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for φ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

PPLTL Validity (φ VAL)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for $\neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

PPLTL Logical Implication ($\Gamma \models \varphi$)

- 1: Given PPLTL formulas Γ and φ
- 2: Compute reverse AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (DFA can be constructed while checking nonemptiness.)
As for LTL_f and LTL on infinite traces.

PPLTL Satisfiability (φ SAT)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for φ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return result of check

PPLTL Validity (φ VAL)

- 1: Given PPLTL formula φ
- 2: Compute reverse AFA for $\neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

PPLTL Logical Implication ($\Gamma \models \varphi$)

- 1: Given PPLTL formulas Γ and φ
- 2: Compute reverse AFA for $\Gamma \wedge \neg\varphi$ (*linear*)
- 3: Compute DFA of reverse of AFA (*exponential*)
- 4: Check DFA for nonemptiness (*NLOGSPACE*)
- 5: Return complemented result of check

Thm: All the above reasoning tasks are PSPACE-complete. (DFA can be constructed while checking nonemptiness.)

As for LTL_f and LTL on infinite traces.

Synthesis from PPLTL Specifications

Given a PPLTL formula φ over a set $\mathcal{P}$ of propositions partitioned into two disjoint sets:

- $\mathcal{X}$ controlled by **environment**
- $\mathcal{Y}$ controlled by **agent**

Find an **agent strategy** σ_a to set the values of $\mathcal{Y}$ in such a way that for all possible values of $\mathcal{X}$, controlled by the **environment**, there exists a k such that the PPLTL formula φ is true in $(X_0, Y_0), \dots, (X_k, Y_k)$.

Algorithm for PPLTL synthesis

- 1: Given PPLTL formula φ
- 2: Compute reversed AFA for φ (linear)
- 3: Compute DFA for reverse of AFA (exponential)
- 4: Synthesize winning strategy for DFA game (linear)
- 5: Return strategy as a transducer

Thm: PPLTL synthesis is EXPTIME-complete.

Exponentially better than for LTL_f and LTL infinite traces!

Synthesis from PPLTL Specifications

Given a PPLTL formula φ over a set $\mathcal{P}$ of propositions partitioned into two disjoint sets:

- $\mathcal{X}$ controlled by **environment**
- $\mathcal{Y}$ controlled by **agent**

Find an **agent strategy** σ_a to set the values of $\mathcal{Y}$ in such a way that for all possible values of $\mathcal{X}$, controlled by the **environment**, there exists a k such that the PPLTL formula φ is true in $(X_0, Y_0), \dots, (X_k, Y_k)$.

Algorithm for PPLTL synthesis

- 1: Given PPLTL formula φ
- 2: Compute reversed AFA for φ (**linear**)
- 3: Compute DFA for reverse of AFA (**exponential**)
- 4: Synthesize winning strategy for DFA game (**linear**)
- 5: Return strategy as a transducer

Thm: PPLTL synthesis is EXPTIME-complete.

Exponentially better than for LTL_f and LTL infinite traces!

- 1 PPLTL: Pure Past LTL
- 2 PPLTL and LTL_f
- 3 PPLTL to Automata
- 4 PPLTL Reasoning and Synthesis
- 5 **PPLTL Again**
- 6 PPLTL to Automata (Revisited)

Weak Since in PPLTL is the correspondent to Weak Until in LTL_f , and Trigger in PPLTL is the correspondent of Release in LTL_f . In Fact Since and Trigger are not much used except for technical reasons (in duals).

BackTo

Weak Since in PPLTL is called “BackTo” and denoted by $\mathcal{B}$ [Manna&Pnueli1992, Chapter 3]. The PPLTL formula $\varphi_1 \mathcal{B} \varphi_2$ says that “ φ_1 holds continuously at all positions preceding (and including) the present, either to the last occurrence of φ_2 , or to position 0”. Formally it is defined as:

$$\varphi_1 \mathcal{B} \varphi_2 \doteq (\varphi_1 \mathcal{S} \varphi_2) \vee \Box \varphi_1$$

Trigger

Trigger denoted by $\mathcal{T}$ is a variant of $\mathcal{B}$ (as release is a variant of weak next). $\varphi_1 \mathcal{T} \varphi_2$ says that “ φ_1 holds continuously at all positions preceding (and including) the present, either to the last occurrence of φ_2 (including it), or to position 0”. It can be defined as:

$$\varphi_1 \mathcal{T} \varphi_2 \doteq \varphi_1 \mathcal{B} (\varphi_1 \wedge \varphi_2)$$

The following holds:

- $\varphi_1 \mathcal{T} \varphi_1 \equiv \neg(\neg \varphi_1 \mathcal{S} \neg \varphi_2)$
- it also holds that
 $\varphi_1 \mathcal{S} \varphi_2 \equiv \neg(\neg \varphi_1 \mathcal{T} \neg \varphi_2)$

(Trigger is **dual** of Since)

Weak Since in PPLTL is the correspondent to Weak Until in LTL_f , and Trigger in PPLTL is the correspondent of Release in LTL_f . In Fact Since and Trigger are not much used except for technical reasons (in duals).

BackTo

Weak Since in PPLTL is called “BackTo” and denoted by $\mathcal{B}$ [Manna&Pnueli1992, Chapter 3]. The PPLTL formula $\varphi_1 \mathcal{B} \varphi_2$ says that “ φ_1 holds continuously at all positions preceding (and including) the present, either to the last occurrence of φ_2 , or to position 0”. Formally it is defined as:

$$\varphi_1 \mathcal{B} \varphi_2 \doteq (\varphi_1 \mathcal{S} \varphi_2) \vee \Box \varphi_1$$

Trigger

Trigger denoted by $\mathcal{T}$ is a variant of $\mathcal{B}$ (as release is a variant of weak next). $\varphi_1 \mathcal{T} \varphi_2$ says that “ φ_1 holds continuously at all positions preceding (and including) the present, either to the last occurrence of φ_2 (including it), or to position 0”. It can be defined as:

$$\varphi_1 \mathcal{T} \varphi_2 \doteq \varphi_1 \mathcal{B} (\varphi_1 \wedge \varphi_2)$$

The following holds:

- $\varphi_1 \mathcal{T} \varphi_1 \equiv \neg(\neg \varphi_1 \mathcal{S} \neg \varphi_2)$
- it also holds that

$$\varphi_1 \mathcal{S} \varphi_2 \equiv \neg(\neg \varphi_1 \mathcal{T} \neg \varphi_2)$$

(Trigger is **dual** of Since)

- $\varphi_1 \vee \varphi_2 \equiv \neg(\neg\varphi_1 \wedge \neg\varphi_2)$ and equivalently $\varphi_1 \wedge \varphi_2 \equiv \neg(\neg\varphi_1 \vee \neg\varphi_2)$
- $\Theta\varphi \equiv \neg\Theta\neg\varphi$ and equivalently $\Theta\varphi \equiv \neg\Theta\neg\varphi$
- $\Box\varphi \equiv \neg\Diamond\neg\varphi$ and equivalently $\Diamond\varphi \equiv \neg\Box\neg\varphi$
- $\varphi_1 \mathcal{T} \varphi_2 \equiv \neg(\neg\varphi_1 \mathcal{S} \neg\varphi_2)$ and equivalently $\varphi_1 \mathcal{S} \varphi_2 \equiv \neg(\neg\varphi_1 \mathcal{T} \neg\varphi_2)$

As for LTL_f

NNF

Negation Normal Form for PPLTL: for $a \in AP$

$$\varphi ::= \text{true} \mid \text{false} \mid A \mid \neg A \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \Theta \varphi \mid \Theta \varphi \mid \Diamond \varphi \mid \Box \varphi \mid \varphi \mathcal{S} \varphi \mid \varphi \mathcal{T} \varphi$$

Theorem

Each PPLTL formula φ admits an equivalent in NNF, denoted $\text{nnf}(\varphi)$, which is obtained in linear time in the size formula by pushing the negation all in exploiting duals.

Exploit duals through the follow equivalences to push negation all the way in:

- $\neg \neg \varphi \equiv \varphi$
- $\neg(\varphi_1 \wedge \varphi_2) \equiv \neg \varphi_1 \vee \neg \varphi_2$
- $\neg(\varphi_1 \vee \varphi_2) \equiv \neg \varphi_1 \wedge \neg \varphi_2$
- $\neg \Theta \varphi \equiv \Theta \neg \varphi$
- $\neg \Theta \varphi \equiv \Theta \neg \varphi$
- $\neg \Diamond \varphi \equiv \Box \neg \varphi$
- $\neg \Box \varphi \equiv \Diamond \neg \varphi$
- $\neg(\varphi_1 \mathcal{S} \varphi_2) \equiv \neg \varphi_1 \mathcal{T} \neg \varphi_2$
- $\neg(\varphi_1 \mathcal{T} \varphi_2) \equiv \neg \varphi_1 \mathcal{S} \neg \varphi_2$

As for LTL_f

NNF

Negation Normal Form for PPLTL: for $a \in AP$

$$\varphi ::= \text{true} \mid \text{false} \mid A \mid \neg A \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \Theta \varphi \mid \Theta \varphi \mid \Diamond \varphi \mid \Box \varphi \mid \varphi \mathcal{S} \varphi \mid \varphi \mathcal{T} \varphi$$

Theorem

Each PPLTL formula φ admits an equivalent in NNF, denoted $\text{nnf}(\varphi)$, which is obtained in linear time in the size formula by pushing the negation all in exploiting duals.

Exploit duals through the follow equivalences to push negation all the way in:

- $\neg \neg \varphi \equiv \varphi$
- $\neg(\varphi_1 \wedge \varphi_2) \equiv \neg \varphi_1 \vee \neg \varphi_2$
- $\neg(\varphi_1 \vee \varphi_2) \equiv \neg \varphi_1 \wedge \neg \varphi_2$
- $\neg \Theta \varphi \equiv \Theta \neg \varphi$
- $\neg \Theta \varphi \equiv \Theta \neg \varphi$
- $\neg \Diamond \varphi \equiv \Box \neg \varphi$
- $\neg \Box \varphi \equiv \Diamond \neg \varphi$
- $\neg(\varphi_1 \mathcal{S} \varphi_2) \equiv \neg \varphi_1 \mathcal{T} \neg \varphi_2$
- $\neg(\varphi_1 \mathcal{T} \varphi_2) \equiv \neg \varphi_1 \mathcal{S} \neg \varphi_2$

As for LTL_f

NNF

Negation Normal Form for PPLTL: for $a \in AP$

$$\varphi ::= \text{true} \mid \text{false} \mid A \mid \neg A \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \Theta \varphi \mid \Theta \varphi \mid \Diamond \varphi \mid \Box \varphi \mid \varphi \mathcal{S} \varphi \mid \varphi \mathcal{T} \varphi$$

Theorem

Each PPLTL formula φ admits an equivalent in NNF, denoted $\text{nnf}(\varphi)$, which is obtained in linear time in the size formula by pushing the negation all in exploiting duals.

Exploit duals through the follow equivalences to push negation all the way in:

- $\neg \neg \varphi \equiv \varphi$
- $\neg(\varphi_1 \wedge \varphi_2) \equiv \neg \varphi_1 \vee \neg \varphi_2$
- $\neg(\varphi_1 \vee \varphi_2) \equiv \neg \varphi_1 \wedge \neg \varphi_2$
- $\neg \Theta \varphi \equiv \Theta \neg \varphi$
- $\neg \Theta \varphi \equiv \Theta \neg \varphi$
- $\neg \Diamond \varphi \equiv \Box \neg \varphi$
- $\neg \Box \varphi \equiv \Diamond \neg \varphi$
- $\neg(\varphi_1 \mathcal{S} \varphi_2) \equiv \neg \varphi_1 \mathcal{T} \neg \varphi_2$
- $\neg(\varphi_1 \mathcal{T} \varphi_2) \equiv \neg \varphi_1 \mathcal{S} \neg \varphi_2$

As for LTL_f

- $\Diamond\varphi \equiv \varphi \vee \Theta(\Diamond\varphi)$ –then choose lfp
- $\Box\varphi \equiv \varphi \wedge \Theta(\Box\varphi)$ –then choose gfp
- $\varphi_1 \mathcal{S} \varphi_2 \equiv \varphi_2 \vee (\varphi_1 \wedge \Theta(\varphi_1 \mathcal{S} \varphi_2))$ –then choose lfp
- $\varphi_1 \mathcal{T} \varphi_2 \equiv \varphi_1 \wedge (\varphi_2 \vee \Theta(\varphi_1 \mathcal{T} \varphi_2))$ –then choose gfp

Fixpoint equations for PPLTL

- $\Diamond\varphi \equiv \varphi \vee \Theta(\Diamond\varphi)$
- $\Box\varphi \equiv \varphi \wedge \Theta(\Box\varphi)$
- $\varphi_1 \mathcal{S} \varphi_2 \equiv \varphi_2 \vee (\varphi_1 \wedge \Theta(\varphi_1 \mathcal{S} \varphi_2))$
- $\varphi_1 \mathcal{T} \varphi_2 \equiv \varphi_1 \wedge (\varphi_2 \vee \Theta(\varphi_1 \mathcal{T} \varphi_2))$

Previous Normal Form

By recursively applying fixpoint equivalences, considering as base case propositions and formulas prefixed with Θ or Θ , i.e.:

$$\begin{array}{ll}
 \text{pnf}(A) & = A \\
 \text{pnf}(\Theta\varphi) & = \Theta\varphi \\
 \text{pnf}(\Box\varphi) & = \Box\varphi \\
 \text{pnf}(\neg\varphi) & = \neg\text{pnf}(\varphi) \\
 \text{pnf}(\varphi_1 \wedge \varphi_2) & = \text{pnf}(\varphi_1) \wedge \text{pnf}(\varphi_2) \\
 \text{pnf}(\varphi_1 \vee \varphi_2) & = \text{pnf}(\varphi_1) \vee \text{pnf}(\varphi_2) \\
 \text{pnf}(\Diamond\varphi) & = \text{pnf}(\varphi) \vee \Theta(\Diamond\varphi) \\
 \text{pnf}(\Box\varphi) & = \text{pnf}(\varphi) \wedge \Theta(\Box\varphi) \\
 \text{pnf}(\varphi_1 \mathcal{S} \varphi_2) & = \text{pnf}(\varphi_2) \vee (\text{pnf}(\varphi_1) \wedge \Theta(\varphi_1 \mathcal{S} \varphi_2)) \\
 \text{pnf}(\varphi_1 \mathcal{T} \varphi_2) & = \text{pnf}(\varphi_1) \wedge (\text{pnf}(\varphi_2) \vee \Theta(\varphi_1 \mathcal{T} \varphi_2))
 \end{array}$$

we get that every formula ψ in PPLTL (analogously to LTL_f) can be decomposed is equivalent to a formula of the form

$$\psi \equiv \text{Bool}(A, \Theta\varphi, \Theta\varphi)$$

that is φ gets partitioned into a part that to be evaluated NOW and a part that to be evaluated at the PREVIOUS instant.

Fixpoint equations for PPLTL

- $\Diamond\varphi \equiv \varphi \vee \Theta(\Diamond\varphi)$
- $\Box\varphi \equiv \varphi \wedge \Theta(\Box\varphi)$
- $\varphi_1 \mathcal{S} \varphi_2 \equiv \varphi_2 \vee (\varphi_1 \wedge \Theta(\varphi_1 \mathcal{S} \varphi_2))$
- $\varphi_1 \mathcal{T} \varphi_2 \equiv \varphi_1 \wedge (\varphi_2 \vee \Theta(\varphi_1 \mathcal{T} \varphi_2))$

Previous Normal Form

By recursively applying fixpoint equivalences, considering as base case propositions and formulas prefixed with Θ or Θ , i.e.:

$$\begin{array}{ll}
 \text{pnf}(A) & = A \\
 \text{pnf}(\Theta\varphi) & = \Theta\varphi \\
 \text{pnf}(\Box\varphi) & = \Box\varphi \\
 \text{pnf}(\neg\varphi) & = \neg\text{pnf}(\varphi) \\
 \text{pnf}(\varphi_1 \wedge \varphi_2) & = \text{pnf}(\varphi_1) \wedge \text{pnf}(\varphi_2) \\
 \text{pnf}(\varphi_1 \vee \varphi_2) & = \text{pnf}(\varphi_1) \vee \text{pnf}(\varphi_2) \\
 \text{pnf}(\Diamond\varphi) & = \text{pnf}(\varphi) \vee \Theta(\Diamond\varphi) \\
 \text{pnf}(\Box\varphi) & = \text{pnf}(\varphi) \wedge \Theta(\Box\varphi) \\
 \text{pnf}(\varphi_1 \mathcal{S} \varphi_2) & = \text{pnf}(\varphi_2) \vee (\text{pnf}(\varphi_1) \wedge \Theta(\varphi_1 \mathcal{S} \varphi_2)) \\
 \text{pnf}(\varphi_1 \mathcal{T} \varphi_2) & = \text{pnf}(\varphi_1) \wedge (\text{pnf}(\varphi_2) \vee \Theta(\varphi_1 \mathcal{T} \varphi_2))
 \end{array}$$

we get that every formula ψ in PPLTL (analogously to LTL_f) can be decomposed is equivalent to a formula of the form

$$\psi \equiv \text{Bool}(A, \Theta\varphi, \Theta\varphi)$$

that is φ gets partitioned into a part that **to be evaluated NOW** and a part that **to be evaluated at the PREVIOUS instant**.

Observations

- LTL_f has a similar normal form obtained from the fixpoint equations. Indeed we have that every formula ψ can be put in “Next Normal Form”, i.e., $\psi \equiv Bool(A, \bigcirc\varphi, \bullet\varphi)$.
- However in the case of LTL_f we have to “**guess**” the value of the formulas of the form $\bigcirc\varphi$ and $\bullet\varphi$
- Instead, for PPLTL the values of the formulas $\ominus\varphi$ and $\ominus\varphi$ is **known**!
- This is the key to the **exponential advantage** of PPLTL wrt LTL_f in computing the DFA.
- We can use this observation to get the DFA in a **factorized/symbolic form in linear time**!

- 1 PPLTL: Pure Past LTL
- 2 PPLTL and LTL_f
- 3 PPLTL to Automata
- 4 PPLTL Reasoning and Synthesis
- 5 PPLTL Again
- 6 PPLTL to Automata (Revisited)

PPLTL: Pure Past LTL

$$\varphi ::= A \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid \Theta\varphi \mid \varphi_1 \mathcal{S} \varphi_2$$

Recall other operators can be seen as abbreviations, e.g., $\Diamond\varphi \equiv \text{true} \mathcal{S} \varphi$; $\Box\varphi \equiv \neg\Diamond\neg\varphi$; $\Theta\varphi \equiv \neg\Theta\neg\varphi$.

Syntactic Closure of an PPLTL formula

The syntactic closure, also called Fisher-Ladner closure, CL_ψ of an PPLTL formula ψ is a set of PPLTL formulas inductively defined as follows:

$$\begin{aligned} \psi &\in CL_\psi \\ \varphi \in CL_\psi &\text{ implies } \neg\varphi \in CL_\psi \\ \neg\varphi \in CL_\psi &\text{ implies } \varphi \in CL_\psi \\ \varphi_1 \wedge \varphi_2 \in CL_\psi &\text{ implies } \varphi_1, \varphi_2 \in CL_\psi \\ \Theta\varphi \in CL_\psi &\text{ implies } \varphi \in CL_\psi \\ \varphi_1 \mathcal{S} \varphi_2 \in CL_\psi &\text{ implies } \varphi_1, \varphi_2, \Theta(\varphi_1 \mathcal{S} \varphi_2) \in CL_\psi \end{aligned}$$

Observe that the cardinality of CL_ψ is linear in the size of ψ .

The result we present here extend immediately if we use all the other temporal operators of PPLTL.

Current and Previous Instants

Given a PPLTL formula ψ over the atomic propositions $\mathcal{P}$:

- let Π_{now} be the propositional interpretation of the atomic formulas $A \in \mathcal{P}$, and
- let Π_{prev} be a propositional interpretation of new propositions “ $\Theta\varphi$ ” for every the subformula of the form $\Theta\varphi$ of CL_ψ .
- let $val(\varphi, \Pi_{now}, \Pi_{prev})$ be the valuation function that evaluated the truth value of ψ , given the propositional interpretations Π_{now} and Π_{prev} of the current and previous instant respectively, defined as follows:

$$\begin{aligned}
 val(A, \Pi_{now}, \Pi_{prev}) &= A \text{ if variable } A \text{ is true in } \Pi_{now} \\
 val(\Theta\varphi, \Pi_{now}, \Pi_{prev}) &= \Theta\varphi \text{ if variable “}\Theta\varphi\text{” is true in } \Pi_{prev} \\
 val(\neg\varphi, \Pi_{now}, \Pi_{prev}) &= \neg val(\varphi, \Pi_{now}, \Pi_{prev}) \\
 val(\varphi_1 \wedge \varphi_2, \Pi_{now}, \Pi_{prev}) &= val(\varphi_1, \Pi_{now}, \Pi_{prev}) \wedge val(\varphi_2, \Pi_{now}, \Pi_{prev}) \\
 val(\varphi_1 \mathcal{S} \varphi_2, \Pi_{now}, \Pi_{prev}) &= val([\varphi_2 \vee (\varphi_1 \wedge \Theta(\varphi_1 \mathcal{S} \varphi_2))], \Pi_{now}, \Pi_{prev})
 \end{aligned}$$

Example

- Consider the PPLTL formula $\psi = \Theta a \wedge (\neg b \mathcal{S} c)$
 - $pnf(\psi) = \Theta a \wedge (c \vee (\neg b \wedge \Theta(\neg b \mathcal{S} c)))$
 - $CL_\psi = \{a, b, c, \Theta a, \Theta(\neg b \mathcal{S} c), \dots, \Theta a \wedge (\neg b \mathcal{S} c)\}$
 - new propositions: $\{\text{"}\Theta a\text{"}, \text{"}\Theta(\neg b \mathcal{S} c)\text{"}\}$
- Consider the trace $\pi = \{a\}, \{a, c\}, \{a, b\}, \{c\}$

Key set of subformulas	Truth value along the trace
"Y(a)"	→○→●→●→●
"Y(¬b S c)"	→○→○→●→○
Formula	Trace
$Y(a) \wedge (\neg b \mathcal{S} c)$	→○→●→○→●

Symbolic DFA

Given a PPLTL formula ψ we can obtain a factorized/symbolic DFA in linear time!

$$\mathcal{A}_\psi = (\mathcal{P}, Q, q_0, Trans, Final)$$

where

- $\mathcal{P}$ set of boolean variables, one for each atomic proposition, representing which propositions are true/false in the current instant;
- Q is the set of boolean variables “ $\Theta\varphi_1$ ”, ..., “ $\Theta\varphi_n$ ”, one component for each subformulas the form $\Theta\varphi$ in CL_ψ , representing which formulas were true/false at the previous instant + one bit flag to record if the state is final. Given a state $q \in Q$ we denote by $\Pi_{prev}(q)$ the truth valuation of the variables of the form “ $\Theta\varphi_1$ ”, ..., “ $\Theta\varphi_n$ ” and by $flag(q)$ the additional bit.
- q_0 is defined as:
 - $\Pi_{prev}^0(q_0) = (false, \dots, false)$ and $flag(q_0) = false$, initially all formulas of the form $\Theta\varphi$ are false and the flag for the final state is false;
- $Trans(q, \Pi_{now})$ is defined as

$$Trans(q, \Pi_{now}) = Trans_{\Theta\varphi_1}(q, \Pi_{now},) \times \dots \times Trans_{\Theta\varphi_n}(q, \Pi_{now}) \times Trans_{flag}(q, \Pi_{now})$$

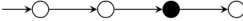
where:

- $Trans_{\Theta\varphi_i}(q, \Pi_{now}) = val(\varphi_i, \Pi_{now}, \Pi_{prev}(q))$, for each variable “ $\Theta\varphi_i$ ”
- $Trans_{flag}(q, \Pi_{now}) \equiv val(\psi, \Pi_{now}, \Pi_{prev}(q))$ for the final flag bit;
- $Final(q)$ iff $flag(q) = true$.

Important: $\mathcal{A}_\psi$ is linear in ψ .

Example (Exercise: define and run the symbolic DFA)

- Consider the PPLTL formula $\psi = \Theta a \wedge (\neg b \mathcal{S} c)$
 - $pnf(\psi) = \Theta a \wedge (c \vee (\neg b \wedge \Theta(\neg b \mathcal{S} c)))$
 - $CL_\psi = \{a, b, c, \Theta a, \Theta(\neg b \mathcal{S} c), \dots, \Theta a \wedge (\neg b \mathcal{S} c)\}$
 - new propositions: $\{\text{"}\Theta a\text{"}, \text{"}\Theta(\neg b \mathcal{S} c)\text{"}\}$
- Consider the trace $\pi = \{a\}, \{a, c\}, \{a, b\}, \{c\}$

Key set of subformulas	Truth value along the trace
" $\Upsilon(a)$ "	
" $\Upsilon(\neg b \mathcal{S} c)$ "	
Formula	Trace
$\Upsilon(a) \wedge (\neg b \mathcal{S} c)$	

Exercise:

- Define symbolic DFA $\mathcal{A}_\psi$
- Run the DFA on the trace π